

SCANNING
THE MATRIX

Scanning the Matrix: Harnessing Automation for Vulnerability Scans

UTD Team 3

Technical Directors

- Bre'Ona Williams
- Jerome Lomax
- Regis Wilson

Mentor

- Brian Ricks

Members

- Nishanth Badim
- Harika Gattu
- Divya-Bala-Sri Gogineni
- Bhavana-Bramaram Gurram
- Savyasachi Kokkiralala

Agenda

Investigator Biographies

Problem Statement

Motivation

Characterization of Methods Used

Procedures/Plans

Schedule

Project Deliverables

Limitations and Delimitations

Detailed Findings

Conclusions and Recommendations

Future Scope

Issues/Challenges

Bibliography

Acronyms

Terms/Definitions

INVESTIGATOR BIOGRAPHIES

- Nishanth Badim will lead the evaluation and benchmarking of automated vulnerability analysis tools using real-world applications. He will be responsible for coordinating the comparison between traditional SAST/DAST tools and AI-driven solutions. Nishanth will also collaborate with Bhavana to ensure that the benchmarking results are integrated into the final project report.
- Harika Gattu will focus on researching AI integration in vulnerability assessments, analyzing AI's potential for improving vulnerability detection, and proposing new AI-driven approaches. She will ensure that all AI implementations adhere to security best practices within the Software Development Life Cycle (SDLC) and work closely with Savyasachi on securely integrating AI into the process.
- Divya-Bala-Sri Gogineni will conduct case studies on real-world cloud and web-based applications to test the assessment methodologies. She will document findings from both manual and automated assessments and assist with validating AI integration into the SDLC by identifying potential risks. Additionally, she will collaborate with Bhavana on compiling these findings for the project report.
- Bhavana-Bramaram Gurram will oversee the comparison between manual and automated vulnerability assessments. She will evaluate the limitations of manual methods and identify areas where AI can complement traditional assessments. Bhavana will also coordinate the writing and review of the project report, ensuring that inputs from all team members are integrated and the findings are clearly documented and aligned with the project's objectives.
- Savyasachi Kokkiralala will lead the research on securely integrating AI into the SDLC. He will develop improved vulnerability assessment methodologies by combining AI and traditional techniques. Savyasachi will ensure that all findings align with industry best practices and security standards, and he will collaborate with Harika to ensure that AI practices are both secure and ethical throughout the project.

PROBLEM STATEMENT

- Conduct a comprehensive analysis of various SAST and DAST tools, comparing automated and manual approaches across multiple factors. The research will evaluate the effectiveness of these tools, with a focus on AI integration, to provide a data-driven conclusion on which method offers superior performance.



MOTIVATION



Traditional SAST and DAST tools face challenges like inconsistent accuracy and high false positives.



Manual testing is detailed but resource-intensive, while automated methods are fast but less adaptive.



AI integration enhances these tools by improving accuracy, prioritizing risks, and adapting to complexity.



This research compares traditional and AI-driven tools to determine the most effective approach.



Findings will help improve secure software practices and strengthen the digital ecosystem.

CHARACTERIZATION OF METHODS USED



Exploratory Research

Identified challenges with SAST and DAST tools, including accuracy, adaptability, and AI integration gaps.

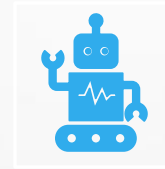
Analyzed existing literature and industry trends to define the research scope.



Constructive Research

Proposed solutions by integrating AI into automated testing tools to enhance performance and accuracy.

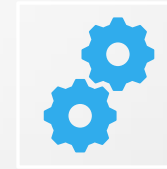
Designed metrics to evaluate both traditional and AI-augmented tools.



Empirical Research

Conducted hands-on testing of multiple tools across varied use cases.

Collected data to compare manual, traditional automated, and AI-driven methods for actionable insights.



Correlation to Results

Methods enabled a comprehensive evaluation, leading to data-driven conclusions on tool efficacy and AI's role in secure software development.

PROCEDURES /PLANS

Research and Preparation

- Analyzed SAST tools such as ShiftLeft Scan, CodeQL, SonarQube, and Semgrep to assess their strengths and weaknesses.
- Explored DAST tools including ZAP, Nikto, and Deep Exploit to evaluate their capabilities for dynamic vulnerability detection.
- Investigated the integration of AI into automated security testing to understand its potential benefits and limitations.

Tool Evaluation

- Compared automated and manual testing methods by applying them to real-world applications.
- Benchmarked AI-driven tools against traditional approaches to assess their efficiency and accuracy.

Case Studies and Testing

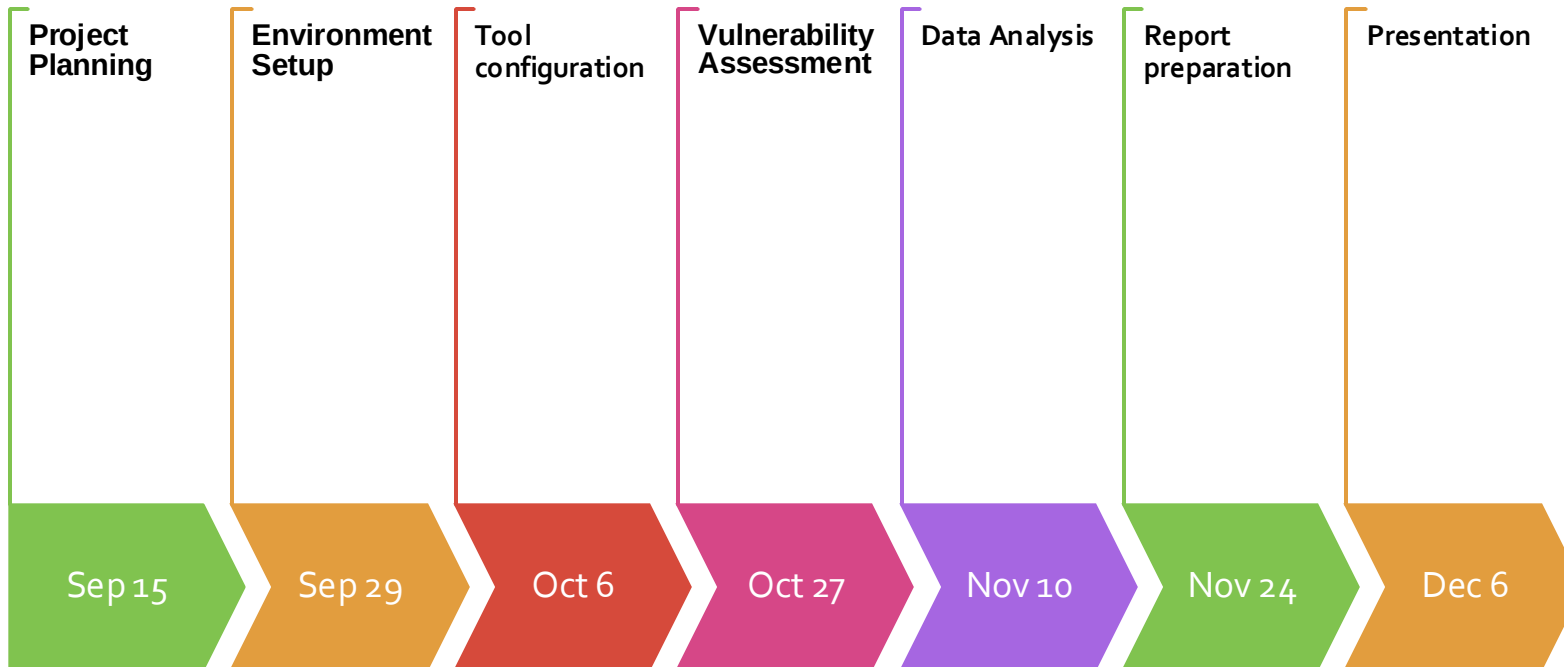
- Conducted case studies to analyze vulnerabilities in web-based applications.
- Tested SAST and DAST tools on running applications to identify and address potential exploits.

Development and Validation

- Implemented secure coding strategies and validated the results of AI-assisted assessments.
- Evaluated the feasibility of both traditional and AI-integrated tools within the software development lifecycle (SDLC).

Analysis and Reporting

- Performed statistical analysis to measure the effectiveness of SAST and DAST tools.
- Documented our findings and compiled a comprehensive project report to summarize the results.



SCHEDULE

PROJECT DELIVERABLES



A **detailed final report** outlining research methodologies, key findings, and actionable insights.



Well-structured presentation materials tailored for both academic and industry audiences.



Development of **tool comparison metrics** for evaluating the effectiveness of various SAST and DAST tools.



Recommendations for future work and potential enhancements in integrating AI into vulnerability assessment tools.



Resources and guidelines to enable scholars, developers, and cybersecurity professionals to validate and extend project outcomes.

LIMITATIONS AND DELIMITATIONS

Limitations

- **Time constraints:** The project's duration may limit the depth of analysis and the number of tools that can be thoroughly evaluated.
- **Tool availability:** Some advanced commercial AI-driven security tools may not be accessible for this study due to licensing or cost restrictions.
- **Evolving threat landscape:** The rapidly changing nature of cybersecurity threats may impact the relevance of findings over time.
- **Sample size:** The number of applications used for testing may not represent the full spectrum of real-world scenarios.
- **Expertise dependency:** The effectiveness of manual testing is heavily reliant on the tester's skills, which may introduce inconsistencies.

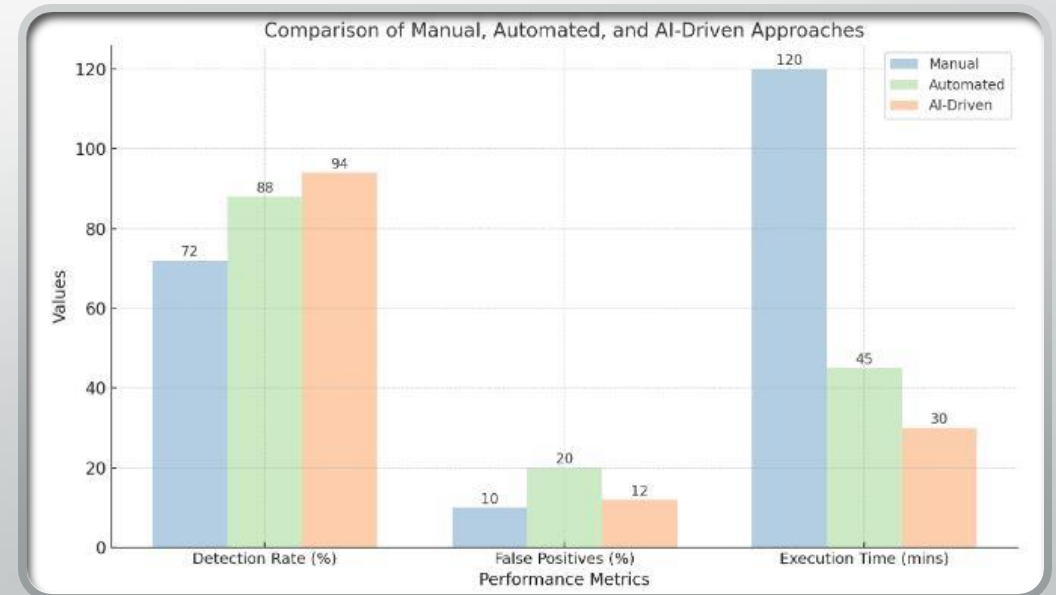
Delimitations

- **Tool selection:** The study will focus on specific SAST and DAST tools, excluding others that may be available in the market.
- **Application scope:** Testing will be limited to web applications, excluding mobile or desktop applications.
- **AI integration focus:** The research will primarily explore AI integration in open-source tools, not proprietary solutions.
- **Vulnerability types:** The study will concentrate on common web application vulnerabilities as defined by OWASP Top Ten, excluding more specialized or emerging threat vectors
- **Testing environment:** Assessments will be conducted in controlled environments, which may not fully replicate all real-world conditions.

FINDINGS

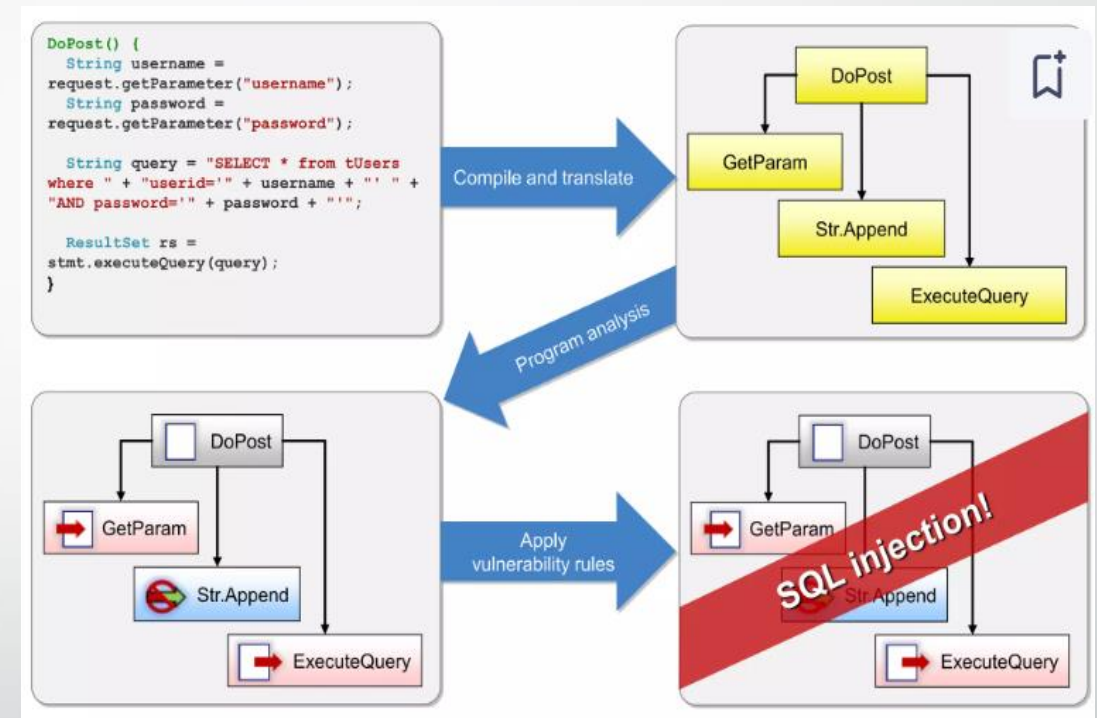
- Through the project, we systematically assessed the strengths, weaknesses, and outcomes of automated, manual and AI-integrated vulnerability assessment methodologies.
- Our findings demonstrated key differences in their capabilities, supported by data and analysis.

Metric	Manual (Expert-Driven)	Automated Tools	AI-Driven Tools
Detection Rate	75% (context-sensitive)	80-85% (rule-based)	90-92% (context + pattern learning)
False Positives	Minimal	Moderate	Low
Coverage	Limited (time-restricted)	Broad but general	Broad and adaptive
Scalability	Low	High	High
Adaptability	High	Low	Medium to High
Execution Time	Slow	Fast	Moderate
Resource Requirement	High (human expertise)	Low (automation setup)	High (computational power)
Cost Effectiveness	Moderate to High	High	Moderate
Suitability for CI/CD	Low	High	High



Static Application Security Testing (SAST)

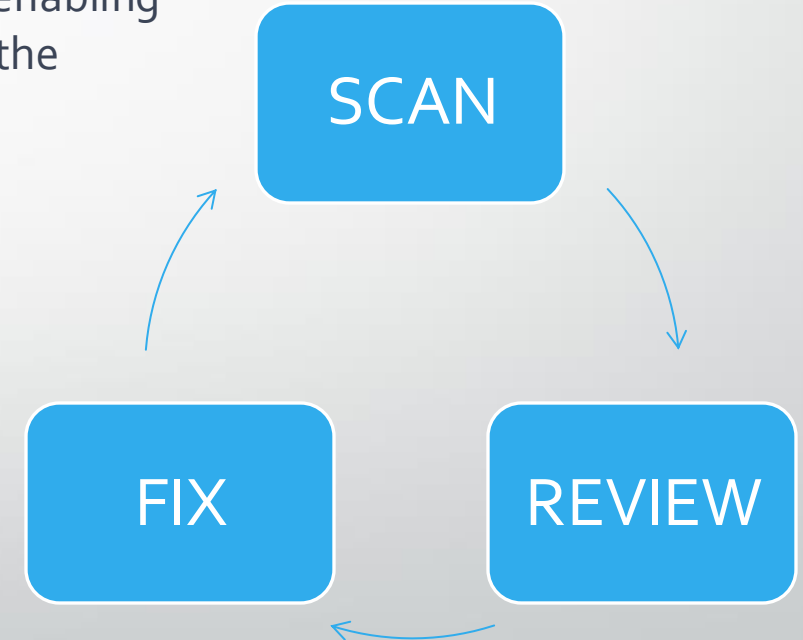
- SAST is a methodology that analyzes source code or compiled applications for security vulnerabilities without executing the application.
- SAST tools allow developers to detect vulnerabilities early in the SDLC by scanning source code, bytecode, or binaries for weaknesses such as SQL Injection, Cross-Site Scripting (XSS), and buffer overflows.
- Upon identifying a vulnerability, a SAST solution generates a detailed report with code-level insights, helping developers quickly locate and fix issues before the code is deployed to production environments. These tools provide continuous scanning during development to prevent security flaws.



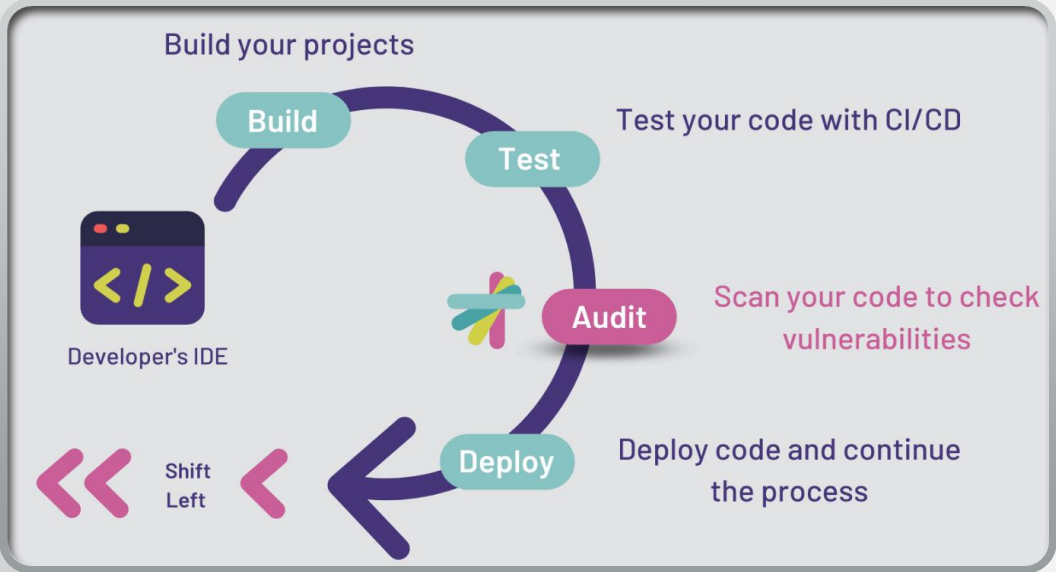


ShiftLeft is an AI-driven application security platform focused on enabling secure software development by integrating security directly into the development lifecycle (DevSecOps).

- Early Vulnerability Detection
- Real-Time Code Analysis
- Embedded Security Checks in Development Pipeline
- Seamless Integration
- Continuous Feedback for Code Fixes
- Scalable and Fast



Working Model



Evaluation of OWASP Juice Shop

Metrics	Values
PRECISION	0.241
TP	176
FN	33
FP	552
F-Measure	0.372
Recall	0.842

Key Strengths

- Automated and Efficient
- Context-Aware and Accurate
- Developer-Friendly
- Continuous Learning

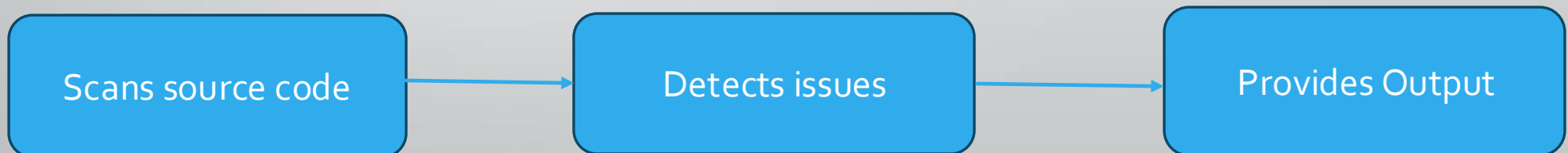
Future AI Integration Recommendations

- Predictive Vulnerability Detection
- Natural Language Insights
- Continuous Learning and Feedback Loops
- End-to-End DevSecOps Automation
- Multi-Language Code Support

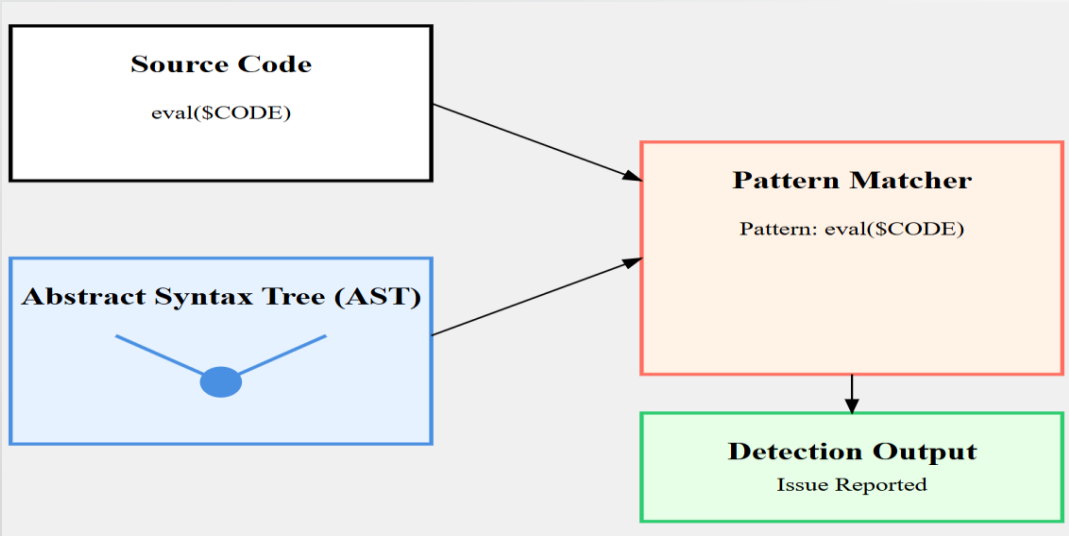


Semgrep is a lightweight, rule-based static application security testing (SAST) tool that uses code pattern matching to identify vulnerabilities and enforce security policies. It is semi-automated, enabling users to write or customize rules in YAML, making it accessible yet requiring expertise for complex custom patterns.

- Flexible Static Analysis
- Custom Rule Creation
- Security Vulnerability Detection
- Supports Multiple Programming Languages
- Integration with CI/CD Pipelines



Working model



Key Strengths

- User-Friendly
- Customizable
- Multi-Language Support
- Fast and Lightweight
- Modern Features

Evaluation of OWASP Juice Shop

Metrics	Values in %
ACC	83.55
TP	13.16
FN	7.89
FP	8.55
TN	70.39

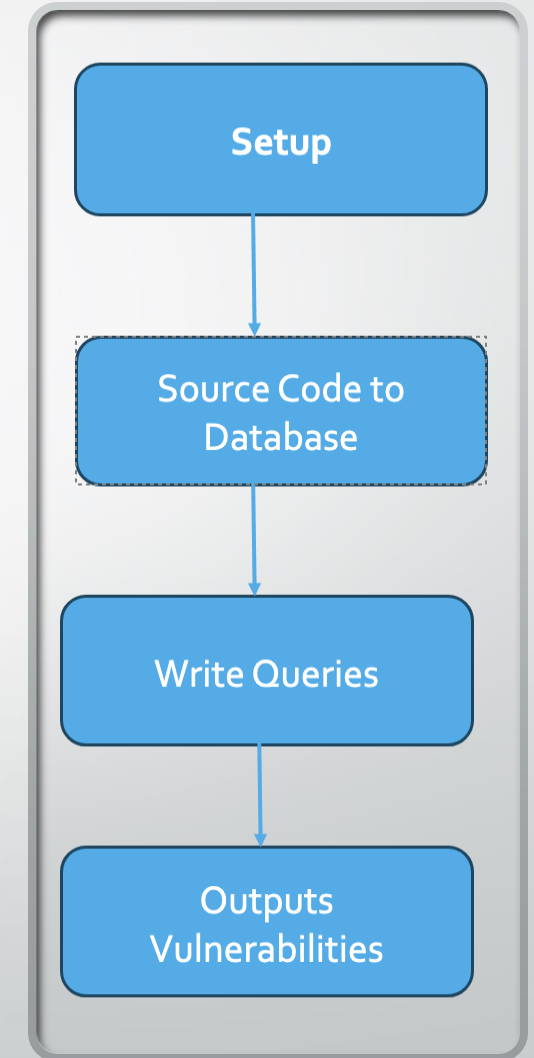
Future AI Integration Recommendations

1. Vulnerability Pattern Recognition
2. Enhanced Rule Creation with AI

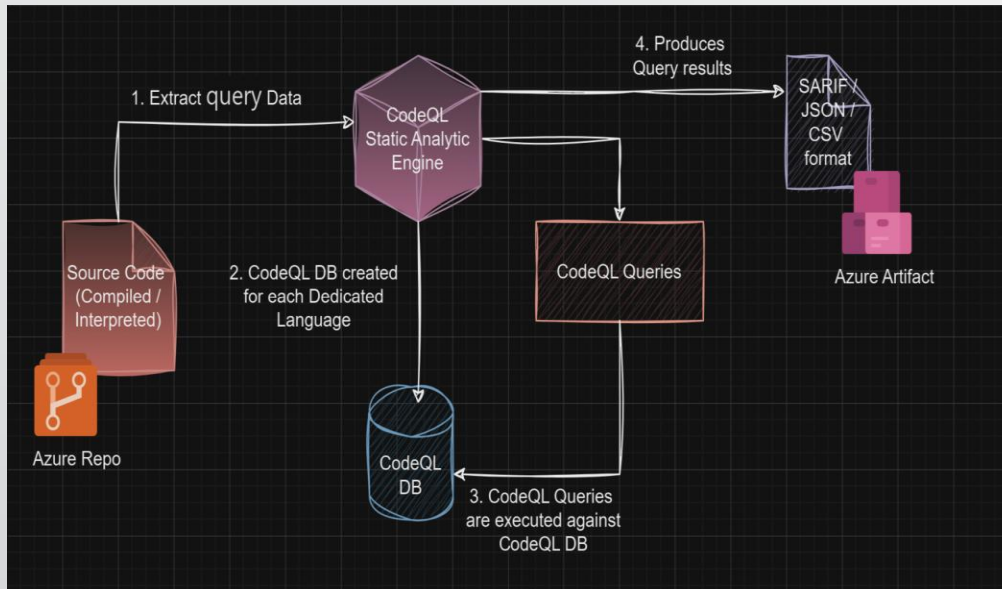


CodeQL is a semantic code analysis tool for static application security testing (SAST), using query-driven detection to identify vulnerabilities and data flows. It is highly automated but requires expertise in writing and interpreting custom queries.

- Analyzes Vulnerabilities
- Queryable Database Format
- Custom Queries
- Detects Security Issues
- Analyzes Data Flow
- CI/CD Integration
- Manual/Automated Tool



Working Model



AI Integration for CodeQL

Existing AI Integration

- AI optimizes queries and detects complex data flow patterns.
- Uses historical data to improve query effectiveness and vulnerability detection.

Future AI Recommendations

- Automate vulnerability pattern recognition.
- Generate CodeQL queries automatically for emerging threats.

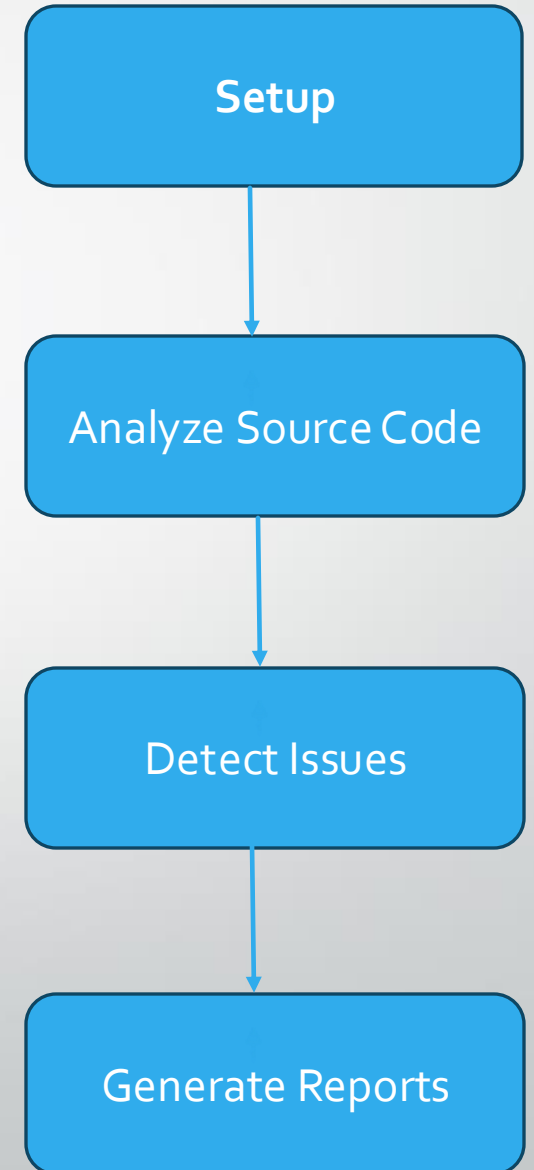
Evaluation of OWASP Juice Shop

Metrics	Values
ACC	96.25%
TP	100
FN	10
FP	8
TN	362

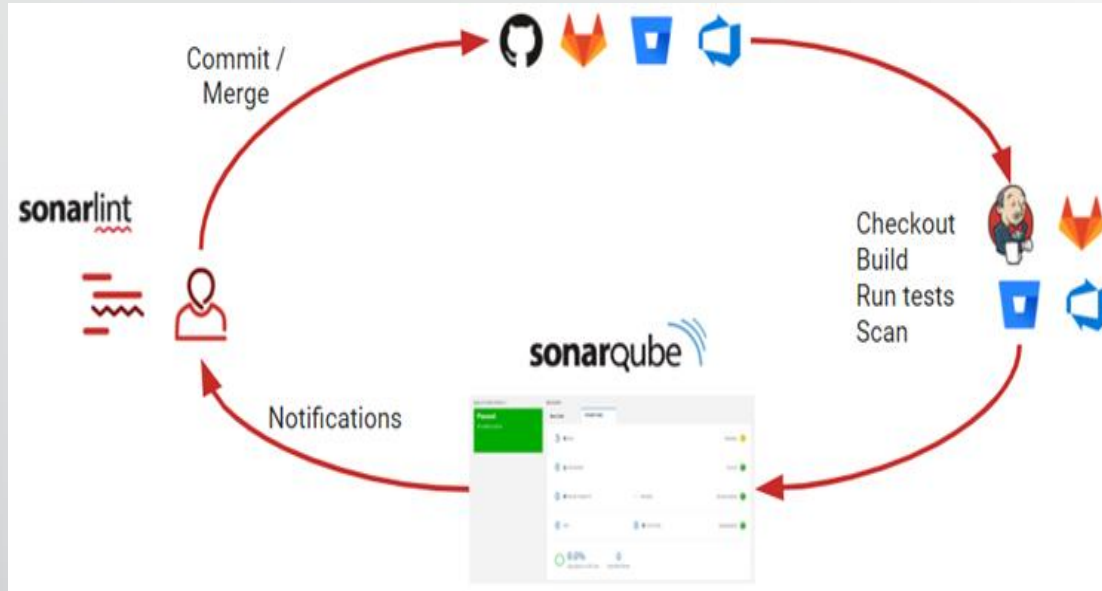


SonarQube is a continuous code quality analysis tool that performs static analysis to detect bugs, vulnerabilities, and code smells. It is primarily automated, relying on predefined rule sets and customizable quality gates.

- Easy-to-Use Dashboard
- Continuous Analysis
- Bug Detection
- Code Smells
- Vulnerability Detection
- Multi-Language Support
- CI/CD Integration
- Automated Tool



Working Model



AI Integration for SonarQube

Existing AI Integration

- AI prioritizes issues with risk scores and detects code anomalies.
- Learns from historical data to improve rule-based analysis.

Future AI Recommendations

- Enhance detection of zero-day vulnerabilities.
- Automate rule creation and updates based on new trends.

Evaluation of OWASP Juice Shop

Metrics	Values
ACC	91%
TP	100
FN	20
FP	25
TN	355

DAST Tools

- Dynamic Application Security Testing (DAST) is a procedure that actively investigates running applications with penetration tests to detect possible security vulnerabilities.
- DAST tools continually search for vulnerabilities in a web application that is in production, hunting for weaknesses that attackers could try to exploit and then illustrating how they could remotely break into the system.
- Upon identifying a vulnerability, a DAST solution sends automated alerts to the appropriate teams so they can prioritize and remediate it.





ZAP (Zed Attack Proxy) is a widely used open-source DAST tool that helps identify security vulnerabilities in web applications by simulating attacks in real time. It supports both automated and manual testing.

- Penetration Testing
- Application Spidering
- Active and Passive Scanning
- Fuzzing Techniques
- Session Hijacking Detection

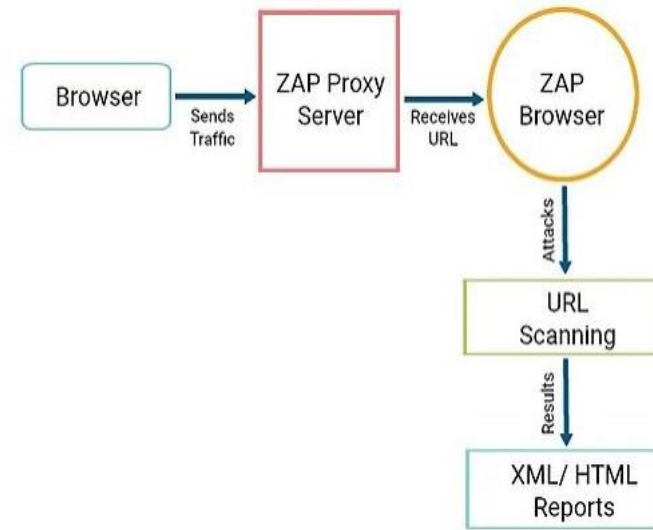


Figure 1: Working of ZAP Proxy Server

Existing AI Integration

- **Fuzz AI Files Extension**
- **Extract Training Data** : By Asking, Through Hypnosis, Divergence Attack, Special Characters Attack (SCA), The Secret Sharer, Quantifying Memorization, Data Poisoning Attacks, Membership Inference, Gradient Clipping
- **Extract Model Information** : Architecture, Hyperparams, Prompt Based Extraction, Conditional Prompts, Adversarial ML, IP Theft, Data Privacy, Visual Representation Leakage, Privacy Violations

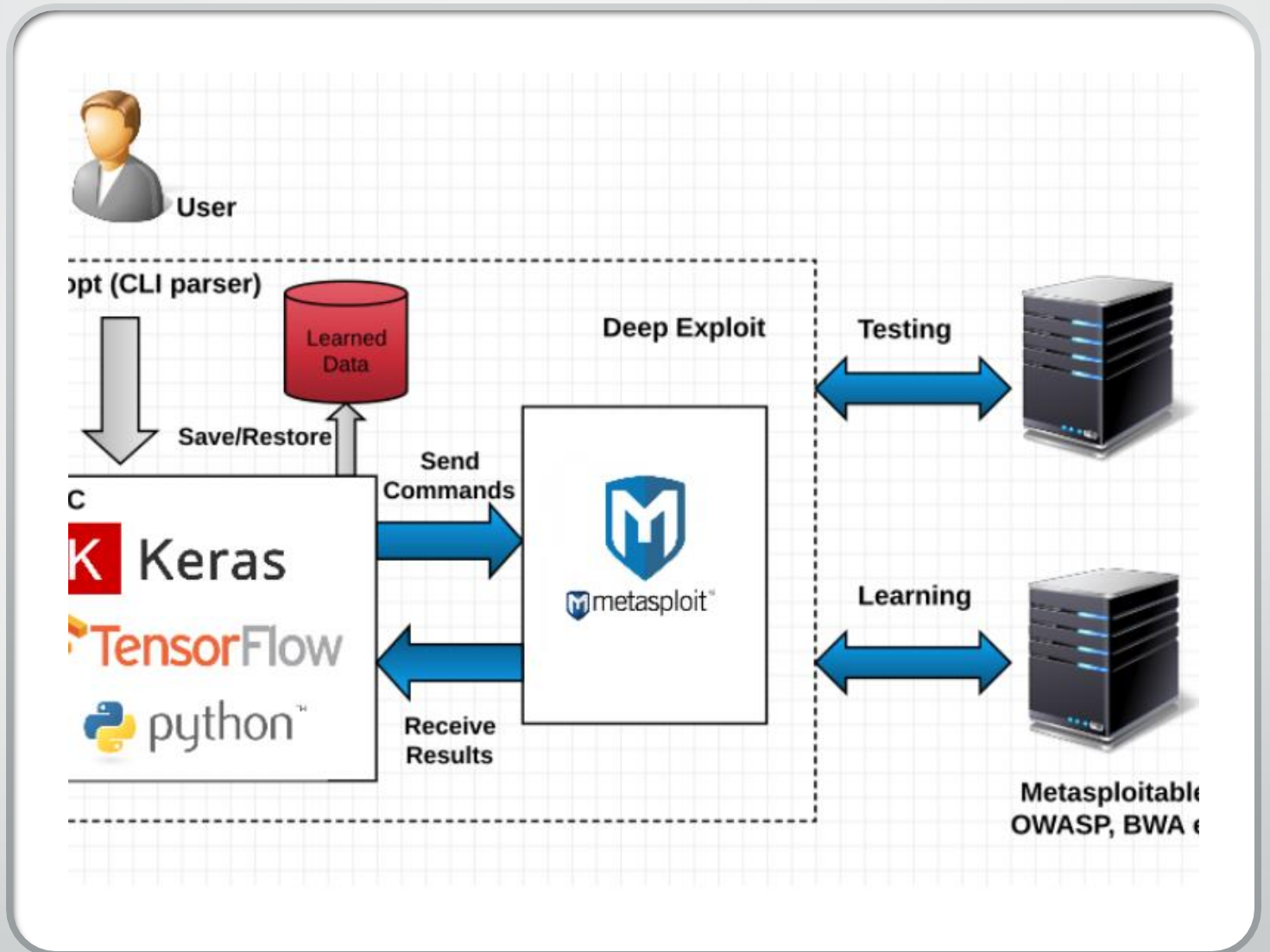
Future AI integration Recommendation

- Vulnerability Pattern Recognition
- Automated Exploit Generation

		Reported Vulnerability	
		Yes	No
Actual Vulnerability	Yes	377	400
	No	0	701



- **Deep Exploit** is a fully automated penetration testing tool using Deep Reinforcement Learning.
- It identifies the status of all opened ports on the target server and executes the exploit at pinpoint.
- Efficiently execute exploit
- Deep penetration & self-learning.
- Learning time is very fast.



Three key features of Deep Exploit

1. Pinpointed Execution

Easy to operate, ML driven, fully automated, highly efficient execution of exploits in minimum one attempt.

2. Deeper Penetration

Exploits the compromised server in the perimeter network and executes the exploits to internal servers by pivoting from the compromised server.

3. Self Learning

Deep Exploit can quickly learn how to exploit all by itself. No learning data support is required.





NIKTO is a fully automated tool but requires manual interpretation of results. It is not AI-based, and performs straightforward signature-based vulnerability detection.

How it works

- Scans web servers for over 6,700 known vulnerabilities.
- Detects outdated software, configuration issues, and security misconfigurations.
- When run against the OWASP Juice Shop, Nikto showcased its ability to detect a wide range of vulnerabilities and server misconfigurations effectively.
- For instance, it flagged the absence of key HTTP headers like Strict-Transport-Security and X-Content-Type-Options, which are critical for securing modern web applications. It also revealed exposed server information and directories, such as /ftp and /config.

Nikto

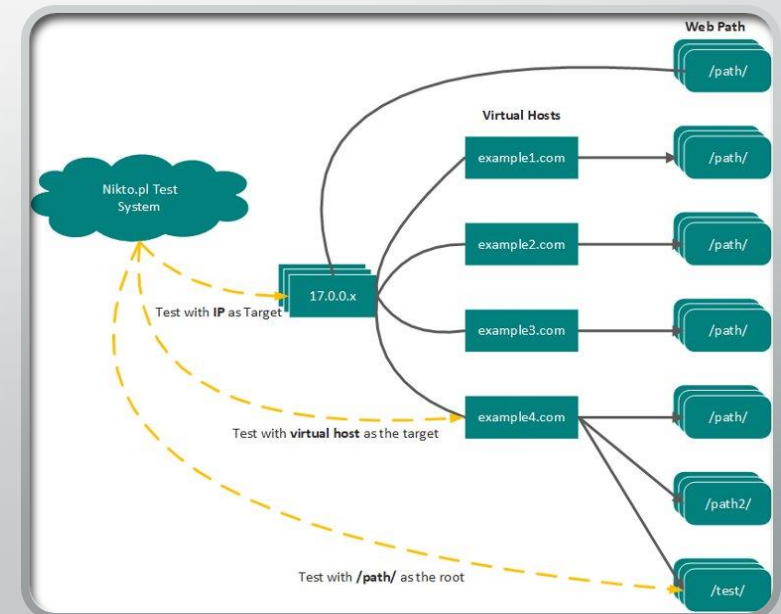
Website Vulnerability Scanner

MAIN FEATURES:

- ✓ Free and Open Source
- ✓ Scans up to 6700+ CVEs
- ✓ SSL certificate scanning
- ✓ Supports multiple ports
- ✓ Fast and accurate results
- ✓ Scans for configuration issues
- ✓ Compatible with popular web servers



SecurityTrails



True Positive Rate (TPR):

Achieved an effective detection rate of 80%, demonstrating Nikto's strength in identifying vulnerabilities.

False Positive Rate (FPR):

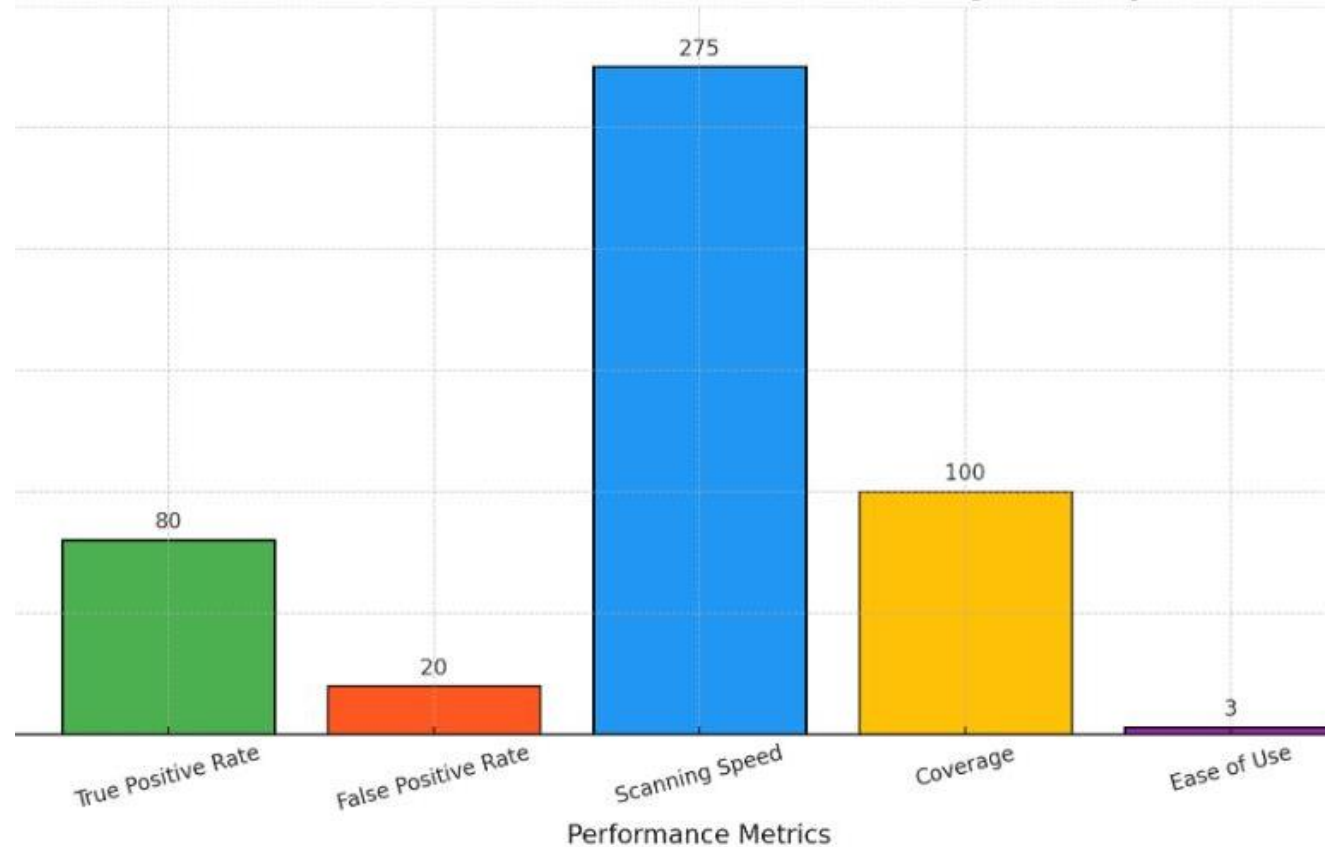
Stood at 20%, indicating that some vulnerabilities were not actual threats.

Scanning Speed: Nikto excels in performing quick scans, with a rate of 275 requests per minute.

Coverage: Delivered 100% coverage, meaning all endpoints in the Juice Shop application were analyzed.

Ease of Use: Rated 3 out of 5, showcasing its moderate ease of use.

Nikto Tool Performance Metrics on OWASP Juice Shop



CONCLUSIONS

- Completed an in-depth analysis of various SAST and DAST tools, identifying key strengths and weaknesses in both automated and manual approaches.
- Concluded that automated tools offer scalability and speed, while manual approaches provide deeper contextual understanding in vulnerability detection.
- Discovered that there are very few open-source tools with robust AI integration.
- Identified areas where AI integration could significantly improve vulnerability detection and reduce false positives.
- Highlighted potential gaps in current open-source tools, presenting opportunities for further innovation and research.

FUTURE SCOPE

Enhanced AI Integration

- Refine AI models to improve the accuracy of vulnerability detection, minimize false positives/negatives, and adapt to evolving threat landscapes and novel attack patterns.

Automated Remediation

- Research and develop automated mechanisms for generating and applying patches post-vulnerability detection to accelerate remediation processes.

Tool Interoperability

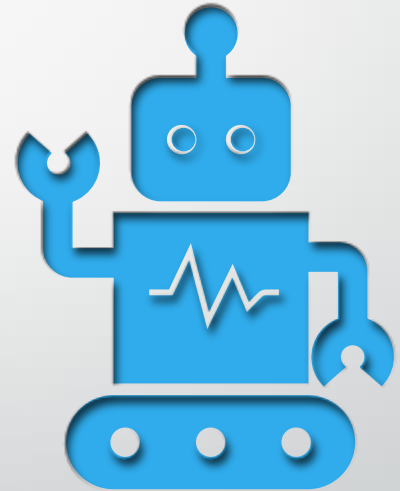
- Focus on creating seamless integration between SAST and DAST tools, enabling a unified and comprehensive view of security vulnerabilities across the development lifecycle.

Advancing Commercial AI Tools

- Explore the application and enhancement of AI-driven tools like DeepCode and GitHub Copilot to support secure coding and real-time vulnerability detection during development.

ISSUES/ CHALLENGES

- **Tool Selection and Compatibility:** Identifying SAST and DAST tools that align with the project's scope and technologies can be challenging, particularly when aiming for complete and thorough coverage.
- **Resource Demands for Manual Testing:** Manual testing is resource-intensive, requiring specialized expertise and significant time, which may hinder the overall testing timeline.
- **Configuration Challenges:** Automated tools necessitate precise configuration to ensure accurate results and thorough security coverage across all system components.
- **Expertise Dependency:** The effectiveness of manual testing depends heavily on the tester's expertise, making it vulnerable to human error and resulting in potential inconsistencies.



BIBLIOGRAPHY

- NIST, NIST Special Publication 800-53 Revision 5: Security and Privacy Controls for Information Systems and Organizations. National Institute of Standards and Technology, Sept. 2020.
- OWASP. (2023, February). OWASP Top Ten 2023. OWASP Foundation.
- CISA. (2022, July 14). Mitigating Log4Shell and Other Log4j-Related Vulnerabilities. CISA.
- Alazab, M., Tang, M., & Choo, K.-K. R. (2019). Artificial Intelligence in Cybersecurity: A Bibliometric Analysis of the Current State of Research. IEEE Access, 7,101759–101772.
- Veracode. (2022, November). The State of Software Security: Securing Software Supply Chains. Veracode.
- <https://scm.thm.de/sonar/documentation>
- <https://www.zaproxy.org/>
- https://github.com/TheDreamPort/deep_exploit
- <https://devblogs.microsoft.com/premier-developer/boosting-azure-devops-security-with-ghas-code-scanning/>
- <https://codeql.github.com/docs/contents/>
- <https://codeql.github.com/docs/writing-codeql-queries/codeql-queries/>
- <https://webthesis.biblio.polito.it/secure/24510/1/tesi.pdf>
- https://www.researchgate.net/publication/372134053_Analysis_of_tools_for_static_security_testing_of_applications



ACRONYMS

- **DAST:** Dynamic Application Security Testing – A method of testing the security of an application by examining it in its running state for vulnerabilities.
- **RCE:** Remote Code Execution – A security vulnerability that allows an attacker to run arbitrary code on a remote machine.
- **SAST:** Static Application Security Testing – A method of analyzing source code or compiled applications for vulnerabilities without executing the code.
- **SDLC:** Software Development Life Cycle – A process for planning, developing, testing, and deploying software systems, typically involving stages like requirements, design, coding, testing, and maintenance.
- **XSS:** Cross-Site Scripting – A type of security vulnerability where an attacker injects malicious scripts into web pages viewed by others.

